

AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER AND SYSTEMS ENGINEERING PROGRAM

RUNE

Low Code Automation Platform

Engineering Core and Academic Thesis (Comprehensive Semester 2 Documentation)

Prepared by:

Seif Elwarwary

Supervised by:

Dr. Yomna Safaaeldin

Eng. Amr Ibrahim

Academic Year 2025–2026

May 27, 2026

Contents

1	System Architecture	1
1.1	High-Level Architecture	1
1.2	Components Overview	1
1.3	Technology Stack	2
1.4	Communication Flow	2
1.5	Design Principles	3
1.6	Repository and Monorepo Layout	3
2	Workflow DSL and Type System	5
2.1	DSL Definition and Versioning	5
2.2	Core DSL Components	5
2.3	Supported Node Types	7
2.4	Parameter Resolution and Expression Evaluation	7
2.5	Error Handling Strategies	7
2.6	Schema Validation	7
2.7	The DSL Generator	8
3	Worker Engine (Go)	9
3.1	Overview and Recursive Executor	9
3.2	Node Registry and Resolver	9
3.3	Control-Flow Nodes	9
3.4	Data-Transformation Nodes	10
3.5	DateTime Family	10
3.6	Integration Node Framework	11
3.7	Agent Node	11
3.8	MCP Support	11
3.9	Observability Hooks in the Worker	11
4	Rust Real-Time Execution Service (RTES)	12
4.1	Overview and Responsibilities	12
4.2	Architecture and Module Decomposition (PR #345)	12

4.3	MongoDB Execution Store	12
4.4	Token Store and Authorization Flow	13
4.5	WebSocket Streaming	13
4.6	Multi-Execution/Workflow Subscription	13
4.7	Known Limitations	14
5	Webhook Trigger Subsystem	15
5.1	Ingress Endpoint	15
5.2	Canvas-Side Webhook Trigger Node	15
5.3	Authentication and Replay Protection	15
5.4	Payload Shape and Resolution into \$input	15
6	Data Flow and Execution Lifecycle	16
6.1	Overview	16
6.2	High-Level Execution Flow	16
6.3	Message and Data Lifecycles	16
6.4	Wait / Resume Semantics	17
6.5	Error Handling and Retry Behavior	17
6.6	Real-Time Updates via RTES	18
6.7	Observability and Auditing	18
7	Workflow Versioning	19
7.1	Motivation	19
7.2	Backend Cutover (PR #396)	19
7.3	Frontend Versioning	20
7.4	Execution Pinning to Versions	20
7.5	Historical Graph Replay in Canvas	20
7.6	Interaction with Templates and Export	20
8	Reliability, Concurrency, and Operations	21
8.1	Save Deduplication and Concurrent Save Prevention	21
8.2	Workflow Status Lifecycle and Computation	21
8.3	RabbitMQ Tuning and Pool Configuration	21
8.4	Migration Bootstrap at API Startup	21
8.5	Failure Modes Catalog	22
	Appendix A: Per-Member Contributions	23

Chapter 1

System Architecture

1.1 High-Level Architecture

The RUNE platform is designed as an asynchronous, event-driven microservices architecture. It separates client-facing REST APIs from the high-throughput, distributed execution backend. The system isolates the processing lifecycle of automated tasks using a message-oriented middleware approach, ensuring that failures in execution instances do not propagate to the HTTP API layer or corrupt persistent configurations.

The system is partitioned into three primary execution tiers:

1. **The Ingress and Control Plane:** Governs workflow design, authorization validation, database mutations, and execution scheduling.
2. **The Transport and Queueing Layer:** Orchestrates routing keys, dead-letter exchanges (DLX), and message persistence.
3. **The Execution and Telemetry Plane:** Consists of stateless workers executing workflow payloads and a Rust WebSocket broker streaming telemetry updates back to the client interface.

1.2 Components Overview

The platform is composed of six distinct services that communicate asynchronously:

- **Backend Master (FastAPI):** The API Gateway and core administrator. Implements user authentication (JWT), RBAC policies, model persistence (using PostgreSQL), credential encryption/decryption, and publishes execution payloads.
- **Worker Engine (Go):** A stateless microservice that consumes execution payloads, compiles execution contexts, parses parameter bindings, runs node-specific code blocks, and evaluates control-flow paths.

- **RTES WebSocket Server (Rust):** A high-performance telemetry server that broadcasts real-time execution states (running, success, failed, waiting) to the visual canvas interface.
- **Archivist (Python):** A background completion processor. It binds to the RabbitMQ completion topic, processes terminal status updates, and updates PostgreSQL database execution tables asynchronously.
- **Scheduler (Python):** A polling cron service that queries PostgreSQL for workflows scheduled to run, triggering runs via protected internal API endpoints.
- **RUNE CLI:** A terminal-based tool built using the Bubble Tea TUI library in Go, supporting operations like database seeds, migration runs, and system diagnostics.

1.3 Technology Stack

The technology stack is selected to optimize concurrency, memory safety, and communication latency:

- **FastAPI (Python 3.13):** Implements the control plane, utilizing async features and Pydantic validation to handle model schema checks.
- **Go Worker Engine:** Compiled to native code, utilizing goroutines and sync pools to support concurrent workflow execution with a minimal memory footprint.
- **Rust RTES:** Built using the tokio runtime and axum WebSocket handlers, ensuring fast execution and memory safety without garbage-collection overhead.
- **RabbitMQ:** The central message broker, supporting task distribution via durable queues.
- **PostgreSQL:** The authoritative relational store, utilizing SQLAlchemy for metadata persistence.
- **Redis:** Handles caching, tracks execution locks, stores single-use WebSocket authorization tokens, and manages Wait-node scheduler timers.
- **MongoDB:** Log persistence store. Saves unstructured, historic execution graphs and telemetry logs.

1.4 Communication Flow

The interaction lifecycle during execution is structured as follows:

1. **Design Phase:** The user defines a workflow on the Next.js visual canvas. The definition is sent via POST to the Backend Master, which validates and stores it in PostgreSQL.
2. **Trigger Phase:** A run is requested via manual trigger, webhook call, or scheduled timer. The Master creates a pending run database entry, resolves environment credentials, and publishes a task to the RabbitMQ `runs` queue.
3. **Execution Phase:** A Go Worker consumes the message, evaluates parameters, runs the target node, updates the execution status via Redis Pub/Sub, and recurses downstream.
4. **Broadcast Phase:** The RTES server captures Redis Pub/Sub events and streams them to the user's browser over a WebSocket.
5. **Archival Phase:** On completion, the worker publishes a final status message to the RabbitMQ `completion` queue. The Archivist consumes this message and saves the final result to PostgreSQL.

1.5 Design Principles

The platform's design relies on four core principles:

- **Stateless Execution:** Workers do not maintain local state. Any crashed worker's task can be safely redelivered to another instance.
- **Contract-Driven API Boundaries:** Frontend API clients and DSL structs are generated from unified schemas, preventing contract drift between the UI, API, and Worker.
- **Isolation of Concerns:** Network outages or database failures in worker instances do not impact API Gateway availability.
- **Deterministic Migrations:** Alembic and database bootstrap routines manage schema mutations deterministically during startup.

1.6 Repository and Monorepo Layout

RUNE is structured as a monorepo, managed via a root-level Makefile:

- `apps/web/`: Next.js frontend application.
- `services/api/`: FastAPI backend service.

-
- `services/rune-worker/`: Go Worker codebase.
 - `services/rtes/`: Rust real-time WebSocket server.
 - `services/archivist/`: Python completion consumer.
 - `services/scheduler/`: Python cron polling service.
 - `dsl/`: Workflow DSL definition files and schema generator script.

Chapter 2

Workflow DSL and Type System

2.1 DSL Definition and Versioning

Workflows in RUNE are modeled using a JSON-based Domain Specific Language (DSL). The schema source of truth is declared in `dsl/dsl-definition.json`. This file defines the structure of all nodes, edges, validation rules, and output variables. DSL definitions are versioned, allowing the system to validate backward compatibility when executing older workflows.

2.2 Core DSL Components

A RUNE DSL document contains three primary structural elements:

- **Workflow Metadata:** Configuration settings (name, description, unique identifier).
- **Nodes Array:** Execution blocks. Each node specifies its type, name, position coordinates, and parameters.
- **Edges Array:** Mappings representing connections between nodes. Each edge identifies a source node, a target node, and optionally an error flag if the edge routes failures.

Listing 2.1: Workflow DSL JSON Example

```
1 {  
2   "id": "wf-100",  
3   "name": "Webhook Notifier",  
4   "nodes": [  
5     {  
6       "id": "node-trigger-1",
```



```
7     "type": "webhook",
8     "name": "Webhook Ingress",
9     "parameters": {
10         "method": "POST"
11     }
12 },
13 {
14     "id": "node-switch-1",
15     "type": "switch",
16     "name": "Filter Action",
17     "parameters": {
18         "expression": "context['$input']['body']['event'] === '
19         alert'"
19     }
20 },
21 {
22     "id": "node-log-1",
23     "type": "log",
24     "name": "Logger",
25     "parameters": {
26         "message": "Received alert: $input.body.message"
27     }
28 }
29 ],
30 "edges": [
31     {
32         "id": "edge-1",
33         "src": "node-trigger-1",
34         "dst": "node-switch-1"
35     },
36     {
37         "id": "edge-2",
38         "src": "node-switch-1",
39         "dst": "node-log-1"
40     }
41 ]
42 }
```

2.3 Supported Node Types

The DSL supports standard execution blocks:

- **Triggers:** Webhook ingress, scheduling timers, and manual HTTP calls.
- **Control-Flow:** Conditional splits, multi-branch switches, synchronization merges, and wait barriers.
- **Data Transforms:** Variable editing, list operations, and datetime parsers.
- **Integrations:** SMTP mailers, Slack notifications, Gmail managers, Google Sheets connectors, and Microsoft Outlook.
- **AI Agent Nodes:** LLM-driven blocks executing tool-use loops.

2.4 Parameter Resolution and Expression Evaluation

Parameters in the DSL can be static values or dynamic references to previous node outputs. Dynamic references use the format `$node_name.path.to.field`. During worker execution, the **Resolver** component parses these paths, fetches the concrete values from the accumulated execution context, and interpolates them before triggering the node. For advanced expressions, users can wrap JavaScript logic in double curly braces, e.g., `{{ $node_name.value*1.15 }}`. This expression is evaluated within a sandboxed Goja JavaScript interpreter inside the Go Worker, preventing malicious system calls.

2.5 Error Handling Strategies

To manage execution failures, nodes can declare one of three error handling behaviors:

- **Halt:** Terminates the entire workflow run immediately and marks the execution as failed.
- **Ignore:** Treats the failure as non-fatal, skips the output injection, and triggers downstream nodes normally.
- **Branch (Error Edges):** Routes execution down a dedicated error edge, allowing users to define compensating tasks or notifications.

2.6 Schema Validation

Before a workflow is saved or executed, the DSL representation is processed by a validation pipeline. The system validates the schema structure against the JSON schema,

checks that all source and target references in edges are valid, verifies that no circular dependency loops exist using topological sorting, and ensures required credentials are configured.

2.7 The DSL Generator

To prevent code drift across services, the platform uses a Python-based generator tool (`dsl/generator/`). The generator parses the primary schema in `dsl-definition.json` and writes:

- Go structs representing inputs, parameters, and outputs.
- TypeScript interfaces for visual node rendering and configuration forms.
- Python Pydantic models for API validation.

CI pipelines validate that no schema changes are committed without running the generator, ensuring consistency.

Chapter 3

Worker Engine (Go)

3.1 Overview and Recursive Executor

The Worker Engine is the core execution runtime of the RUNE platform. Written in Go, it runs as a stateless consumer fleet listening on RabbitMQ queues. Rather than relying on a centralized coordinator that polls execution steps, the Worker uses a recursive execution model. When a task message is pulled from RabbitMQ, the worker instantiates a local `ExecutionContext`, parses parameter expressions, executes the node's handler, modifies the context with the outputs, and evaluates downstream routing. If multiple successor nodes exist, the worker publishes separate `NodeExecutionMessages` to the message bus, fanning out execution concurrently.

3.2 Node Registry and Resolver

The Worker registers constructors for all node types in a centralized `Registry` (`pkg/registry/init_registry.go`) during initialization. During execution, the worker resolves dynamic parameters against the accumulated execution context using the `Resolver` (`pkg/resolver/resolver.go`). The resolver parses path expressions, evaluates bracket notations, and resolves values recursively.

3.3 Control-Flow Nodes

Control-flow nodes coordinate branching, synchronization, and delays:

- **Switch Node:** Directs execution to different paths by evaluating condition predicates or JavaScript expressions in a Goja VM.
- **Split Node:** Loops over an input array and triggers parallel executions of downstream branches by fanning out separate messages.

- **Merge Node:** Syncs fanned-out runs. In `wait_for_all` mode, it uses atomic Redis Lua script decrements to determine when all branches have finished before continuing downstream.
- **Wait Node:** Halts execution by writing state to Redis and setting a timer. When the timer expires, the external Scheduler publishes a resume event.

The Merge Node uses an atomic Lua script executed on Redis to track branch completion without race conditions:

Listing 3.1: Redis Lua Script for Wait-For-All Merge Coordination

```
1 -- KEYS[1]: barrier_key ({prefix}:barrier)
2 -- ARGV[1]: incoming_parent_id
3 -- ARGV[2]: incoming_payload_json
4 -- ARGV[3]: expected_parent_count
5
6 local barrier_key = KEYS[1]
7 local arrivals_key = barrier_key .. ":arrivals"
8 local data_key = barrier_key .. ":data"
9
10 -- 1. Register Arrival
11 redis.call('SADD', arrivals_key, ARGV[1])
12
13 -- 2. Store Payload
14 redis.call('HSET', data_key, ARGV[1], ARGV[2])
```

3.4 Data-Transformation Nodes

Data-transformation nodes let users edit values inside the execution context:

- **Edit Node:** Evaluates JavaScript code within the Goja sandbox to let users filter, map, or restructure keys in the accumulated context.
- **List operations:** Provides standard list operations (filter, sort, map, slice) natively, avoiding the need for custom scripting.

3.5 DateTime Family

The DateTime node family includes specialized blocks for parsing string dates, calculating durations, formatting timestamps, and comparing times. This separation prevents validation errors and ensures date calculations conform to standard UTC representations before integration requests are dispatched.

3.6 Integration Node Framework

RUNE uses a structured Integration Node Framework (`pkg/integrations`) to coordinate connections to third-party APIs. The framework handles OAuth2 token refreshing, request serialization, retry loops, and error mapping. Implemented integrations include Gmail, Google Sheets, and Microsoft Outlook.

3.7 Agent Node

The Agent Node integrates large language models directly into workflows using the Google ADK Go library. It supports tool binding, allowing the agent to evaluate context fields and determine whether it needs to invoke other workflow steps as tools. It features execution guards (token budgets, timeout constraints, loop counters) to prevent runaway execution costs.

3.8 MCP Support

To support local execution tooling, the Go worker implemented experimental Model Context Protocol (MCP) integration, allowing LLMs to execute system tools directly. However, the initial implementation was reverted due to high connection overhead, execution delays, and security concerns with running unverified local commands. The platform is currently being refactored to support a containerized, isolated sandbox model for local tools.

3.9 Observability Hooks in the Worker

The Worker Engine provides deep observability hooks during node runs:

- **Metrics:** Logs queue delays, node execution durations, and error rates via Prometheus metrics.
- **Structured Logs:** Emits structured JSON logs containing execution, workflow, and node identifiers.
- **Traces:** Propagates OpenTelemetry context headers across processes to map the complete execution lifecycle.

Chapter 4

Rust Real-Time Execution Service (RTES)

4.1 Overview and Responsibilities

The RUNE Token Execution Service (RTES) is a WebSocket service written in Rust. It serves as the bridge for streaming execution updates between backend worker processes and the user's browser. RTES maintains persistent connections to clients, receiving execution updates over Redis Pub/Sub and broadcasting them to the frontend canvas.

4.2 Architecture and Module Decomposition (PR #345)

To support codebase maintainability, RTES was refactored from a monolithic file into modular components:

- `src/api/ws.rs`: Upgrade handlers and WebSocket write loops.
- `src/api/handlers.rs`: REST endpoints for health checks.
- `src/infra/execution_store.rs`: MongoDB client interface.
- `src/infra/token_store.rs`: Redis token validation checks.
- `src/infra/messaging.rs`: Pub/Sub consumer loops.

4.3 MongoDB Execution Store

RTES writes live execution states to MongoDB (`infra/execution_store.rs`). When an execution starts, the workflow definition is persisted. As workers run nodes, RTES

updates the database documents with inputs, outputs, status codes, and execution durations. MongoDB was selected for this store due to its document-oriented model, which handles varying node outputs without schema locks.

4.4 Token Store and Authorization Flow

To secure the WebSocket endpoints, the service uses an ephemeral token validation flow managed via Redis:

1. The client requests an execution access token from the API Master.
2. The Master generates a random UUID token, caches it in Redis with a 60-second TTL, and returns it to the client.
3. The client initiates a WebSocket connection to RTES, passing the token as a query parameter.
4. RTES verifies and deletes the token from Redis in a single check, upgrading the connection.

4.5 WebSocket Streaming

When a client establishes a connection, RTES runs a two-phase initialization sequence:

- **Replay Phase:** RTES queries MongoDB for all existing status records related to that execution. It replays these updates to the client to draw the current execution state on the visual canvas.
- **Live Phase:** RTES registers the client connection against the Redis Pub/Sub channel for that execution ID, streaming live updates as they are published by workers.

4.6 Multi-Execution/Workflow Subscription

The updated WebSocket handler supports multiplexing, allowing a single connection to subscribe to multiple execution IDs or workflow IDs. This capability allows the dashboard to display status indicators for all running workflows without opening separate connections.

4.7 Known Limitations

Because visual canvas nodes update linearly, parallel updates from split branches can cause state visual races on the frontend. RTES captures and stores all parallel executions under their lineage hashes in MongoDB, but real-time linear rendering remains limited.

Chapter 5

Webhook Trigger Subsystem

5.1 Ingress Endpoint

RUNE exposes public HTTP ingress endpoints under `/api/v1/webhooks/{webhook_guid}` via the FastAPI Gateway. These endpoints support standard HTTP methods (GET, POST, PUT, DELETE). When an external request is received, the gateway identifies the corresponding workflow from the GUID, validates the request, and queues a new execution.

5.2 Canvas-Side Webhook Trigger Node

The Webhook Trigger Node represents the entry point on the visual canvas. It displays the unique webhook URL, shows format configuration parameters, and lets users specify required authentication keys and headers.

5.3 Authentication and Replay Protection

Ingress webhooks support signature verification (such as HMAC-SHA256 headers) to validate incoming payloads. To prevent replay attacks, the gateway validates request timestamps against a configured validity window (e.g., 5 minutes) and caches unique request signatures, rejecting duplicate payloads.

5.4 Payload Shape and Resolution into `$input`

When a webhook payload is verified, the gateway normalizes the headers, query parameters, and body into a structured JSON payload. This payload is stored in the workflow execution context under the variable `$input`. The worker can then reference these values downstream (e.g., using `$input.body.issue.number`).

Chapter 6

Data Flow and Execution Lifecycle

6.1 Overview

The execution lifecycle tracks how data and control states transition from workflow initiation to completion. RUNE uses an asynchronous, message-driven data model to handle this process.

6.2 High-Level Execution Flow

An execution transitions through the following phases:

1. **Triggering:** A run is requested via manual click, scheduler timer, or webhook ingress.
2. **Queueing:** The API Master compiles the initial context and publishes a task to RabbitMQ.
3. **Worker Execution:** The worker consumes the task, executes the node, updates Redis with the status, and evaluates downstream nodes.
4. **Live Telemetry:** RTES reads the status from Redis and streams it to the client over a WebSocket.
5. **Completion:** Once the final node finishes, the worker publishes a completion message. The Archivist reads this message and updates PostgreSQL.

6.3 Message and Data Lifecycles

The platform defines three canonical message schemas:

- **NodeExecutionMessage**: Passed between workers, containing the workflow definition, target node, accumulated context, and branch lineage hashes.
- **NodeStatusMessage**: Lightweight progress updates published to Redis.
- **CompletionMessage**: Marks execution completion and details terminal states and final contexts.

Listing 6.1: NodeExecutionMessage JSON Payload Example

```
1 {  
2   "execution_id": "exec-abc-123",  
3   "workflow_id": "wf-100",  
4   "workflow_version": 1,  
5   "node_id": "node-switch-1",  
6   "accumulated_context": {  
7     "$input": {  
8       "body": {  
9         "event": "alert",  
10        "message": "CPU threshold exceeded"  
11      }  
12    }  
13  },  
14  "lineage_stack": []  
15 }
```

6.4 Wait / Resume Semantics

When a workflow hits a Wait node, the Go Worker persists the current context to Redis and schedules a timer. The Scheduler service polls these timers and, upon expiration, publishes a resume payload to the `workflow.resume` queue. A dedicated consumer picks up the message, rehydrates the context, and schedules the downstream node.

6.5 Error Handling and Retry Behavior

If a node fails, the executor evaluates the node's error policy. If the failure is transient (e.g., database network blip), the worker rejects the message, causing RabbitMQ to requeue it. Persistent failures are routed to dead-letter queues (DLQ) for investigation.

6.6 Real-Time Updates via RTES

RTES subscribes to status channels on Redis Pub/Sub. When a worker publishes an update, RTES broadcasts the payload to all WebSocket connections subscribed to that execution ID.

6.7 Observability and Auditing

Observability is built into the data flow. The system records structured logs, traces execution calls across service boundaries, and maintains audit logs of execution results in PostgreSQL for compliance.

Chapter 7

Workflow Versioning

7.1 Motivation

As workflows are deployed to run mission-critical tasks, editing them directly can break running instances. RUNE implements a versioning system that separates drafts from active production runs.

7.2 Backend Cutover (PR #396)

The versioning backend splits the workflow data model in PostgreSQL. Workflows are edited as drafts. When the user publishes, a new immutable version is cut. The database schema uses SQLAlchemy definitions:

Listing 7.1: SQLAlchemy Schema representing Workflow and WorkflowVersion models

```
1 class Workflow(SQLModel, table=True):
2     __tablename__ = "workflows"
3     id: UUID = Field(default_factory=uuid4, primary_key=True)
4     name: str = Field(nullable=False)
5     description: Optional[str] = Field(default=None)
6     created_at: datetime = Field(default_factory=datetime.utcnow)
7     active_version_id: Optional[UUID] = Field(default=None,
8         foreign_key="workflow_versions.id")
9
10 class WorkflowVersion(SQLModel, table=True):
11     __tablename__ = "workflow_versions"
12     id: UUID = Field(default_factory=uuid4, primary_key=True)
13     workflow_id: UUID = Field(foreign_key="workflows.id", index=True)
14     version_number: int = Field(nullable=False)
```

```
14     definition: dict = Field(default_factory=dict, sa_column=
15         Column(JSON))
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

7.3 Frontend Versioning

The frontend canvas allows users to browse version histories, compare drafts against previous published versions, and roll back to older snapshots.

7.4 Execution Pinning to Versions

When an execution starts, RUNE pins it to the current published version's snapshot ID. This ensures that edits made to the draft during execution will not affect active runs, preventing state corruption.

7.5 Historical Graph Replay in Canvas

When a user views an execution log, the canvas reads the pinned version ID and re-constructs the exact graph layout as it existed when the run started, preventing rendering errors.

7.6 Interaction with Templates and Export

Exported workflow files include version stamps. When imported, the API Master validates the schema version and seeds the workflow as a new version history, preserving draft states.

Chapter 8

Reliability, Concurrency, and Operations

8.1 Save Deduplication and Concurrent Save Prevention

To prevent conflicts when multiple users edit a workflow simultaneously, RUNE uses optimistic concurrency control. Save requests include the last retrieved version ID. If a write conflict is detected, the API Gateway rejects the save, prompting the user to resolve differences.

8.2 Workflow Status Lifecycle and Computation

The platform computes workflow statuses by aggregating active execution histories. Statuses transition through states like `draft`, `active`, `paused`, and `archived`, which are updated automatically by the API Master.

8.3 RabbitMQ Tuning and Pool Configuration

To handle high execution throughput, the RabbitMQ channels are tuned with prefetch limits (e.g., 50 messages per worker). This configuration prevents single worker instances from hogging tasks and manages back-pressure during load bursts.

8.4 Migration Bootstrap at API Startup

At startup, the API Master verifies the PostgreSQL database schema status. If pending migrations are found, the service prints a warning and blocks initialization until they

are applied, preventing runtime schema mismatches.

8.5 Failure Modes Catalog

The platform defines failure modes and fallback actions:

- **Worker Crash:** RabbitMQ detects connection drops, reclaims unacknowledged tasks, and routes them to another worker.
- **Redis Down:** Wait nodes fail to schedule. The system logs the failure and reroutes tasks to the dead-letter queue.
- **API Gateway Down:** Telemetry updates queue up in Redis until the API Gateway recovers and resumes writes.

Appendix A: Per-Member Contributions

A.5 Seif Elwarwary

Seif Elwarwary is the primary architect and developer of the real-time systems and control-flow execution layers. His contributions include:

- **Real-Time Telemetry (RTES):** Designed and implemented the complete RTES microservice in Rust, using axum and tokio. Developed the Redis Pub/Sub integration and MongoDB execution store (`infra/execution_store.rs`), including the load-then-live history replay mechanism.
- **Worker Control-Flow:** Implemented the recursive executor and developed node execution handlers in Go for Switch, Split, Merge (with Redis-backed Lua scripts), Wait, and Goja-based Edit nodes.
- **Webhook Ingress:** Authored the FastAPI webhook subsystem, including request authentication, signature validation, and entry payload parsing.
- **Versioning Backend:** Implemented the backend cutover to support draft vs. published versions and execution pinning.
- **Key Pull Requests:** PR #259 (Initial RTES service), PR #345 (RTES refactoring and multi-subscription), PR #396 (Versioning backend cutover), PR #593/594 (Webhook trigger integration).